INDEX

S No.	Topics	Page No.
1	Introduction	1
2	Objectives	1-2
3	Components used	2
4	Working	2-4
5	Circuit connections	4-5
6	Code	5-7
7	Challenges faced	7-8
8	Applications	8
9	Future scope	9
10	Conclusion	9-10

DETECTION OF HEART RATE using Arduino

Introduction:

Heart rate monitoring is an essential aspect of healthcare and fitness tracking, used to assess a person's overall cardiovascular health. With advancements in embedded systems and sensor technology, heart rate measurement can now be implemented using microcontrollers like Arduino along with a pulse sensor.

This project focuses on detecting heart rate (beats per minute or BPM) in real-time using an Arduino-based system. The system processes data from a pulse sensor, displays the BPM on an LCD screen, and can trigger an alert if the heart rate exceeds a predefined threshold. This type of system can be useful for fitness enthusiasts, medical monitoring, and IoT-based health applications.

Objectives:

1. Heart Rate Measurement Using a Pulse Sensor:- The project aims to use a pulse sensor that operates on the principle of photoplethysmography (PPG) to detect blood volume changes in the fingertip or earlobe and measure heartbeats.

2. Data Processing Using Arduino:- The system will process the raw analog data from the pulse sensor using an Arduino microcontroller, filter out noise, and calculate the heart rate in beats per minute (BPM).

3. Real-Time Display on LCD Screen:- A 16x2 LCD with I2C module will be used to display the measured heart rate in real time, allowing users to monitor their pulse easily.

4. Alert System for Abnormal Heart Rate:- The system will trigger **a** LED indicator and a buzzer if the heart rate exceeds or falls below a predefined threshold, alerting the user about potential abnormalities.

5. Developing a Cost-Effective and Portable Solution:- This project focuses on building a low-cost, easy-to-use, and portable heart rate monitoring system that can be used in fitness tracking, home healthcare, and clinical applications.

6. Efficient Alert System:- To design a simple, effective alert system using LEDs to indicate sensor readings.

7. Exploring Future Enhancements:- The project may also explore additional features, such as:

- Data logging to store heart rate readings over time.
- Wireless connectivity (WiFi/Bluetooth) for remote monitoring.

- Integration with mobile applications for tracking heart rate history.

Components Used:

1. Arduino Uno: The main microcontroller that processes input from sensors and controls the output devices.

2. Pulse Sensor : detecting changes in blood volume in arteries using light, typically infrared or green, and measuring the amount of light reflected or absorbed by the skin.

3. LED: Indicate the status of the pulse sensor.

4. Breadboard: Used to connect and organize all the components in a structured manner.

5. Jumper Wires: For making secure and reliable electrical connections between components.

6. Resistors (220Ω): Limit current to the LEDs, preventing damage.

7. USB Cable: Powers the Arduino from a laptop or USB power source.

8.LCD Display: Displays the output on the screen .

System Design and Working:

System Design:

The system design of the Arduino-based heart rate detection project involves both hardware components and software algorithms that work together to measure and display heart rate accurately. This section outlines the architecture, components, circuit connections, and working principles of the system.

based on the sensor readings.

Working:

1. Power Supply: The Arduino is powered via a USB connection to a laptop.

2. Pulse Sensor: Detects heart rate .

When it senses senses the heart rate , it sends a signal to the Arduino. (connected to A0).

3. LCD Screen Display:

Displays the output .

The input is sent to the pulse sensor ,which is processed by the arduino microcontroller and the output is displayed by the LCD screen.(SDA connected to A4,SCL connected to A5).

4. Arduino Processing: The Arduino continuously monitors the sensor inputs and controls the LEDs based on the readings.

5. LED Indicators: if the input exceeds the threshold value the LED starts blinking.

Connections:

1. PULSE Sensor Connection:

VCC → +ve rail (5V) on the breadboard

GND → -ve rail (GND) on the breadboard

OUT → connected to analog pin A0

2. LED (on Breadboard):

Long leg (Anode) → D9

Short leg (Cathode) → Gnd

220Ω resistor from short leg of LED then to GND

3. LCD Display Connection:

VCC → +ve rail (5V) on the breadboard

GND → -ve rail (GND) on the breadboard

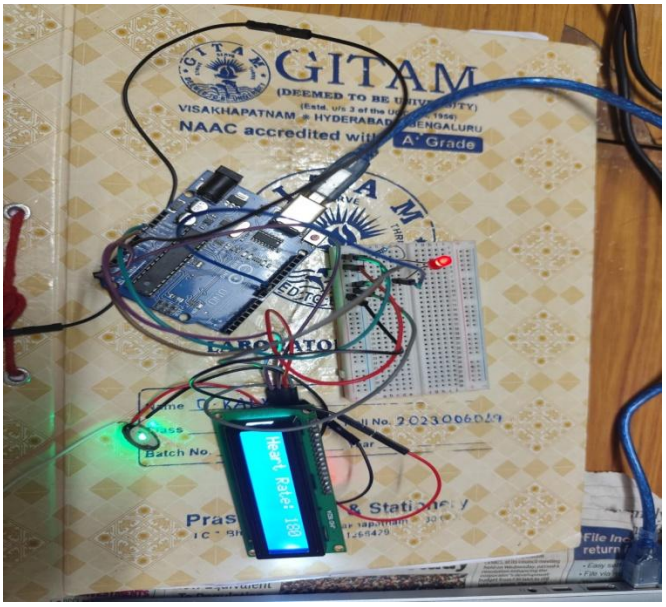
SDA→ connected to the anlog pin(A4).

SCL→connected to the analog pin(A5).

5. Breadboard Power and Ground:

+ve rail (5V) on breadboard → 5V (Arduino)

-ve rail (GND) on breadboard → GND (Arduino)



Code:

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// Initialize the LCD
LiquidCrystal_I2C lcd(0x27, 16, 2); // Change the address if needed

// Pulse Sensor
const int pulsePin = A0; // Pulse sensor signal pin
const int ledPin = 9; // LED pin
int threshold = 200; // Set your heart rate threshold
int heartRate = 0; // Variable to store heart rate
int lastBeatTime = 0; // Time of the last beat
int beatCount = 0; // Count of beats
unsigned long lastTime = 0; // Last time we updated the display

// Moving average variables
const int numReadings = 10; // Number of readings for moving average
int readings[numReadings]; // Array to hold readings
int readIndex = 0; // Current index in the array
int total = 0; // Total of the readings
int average = 0; // Average of the readings

// Baseline check variables
bool baselineEstablished = false;
```

```
int baselineValue = 0;

void setup() {
  Serial.begin(9600);
  lcd.begin(16, 2); // 16 columns and 2 rows
  lcd.backlight();

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Turn off LED initially

  // Initialize readings array
  for (int i = 0; i < numReadings; i++) {
    readings[i] = 0;
  }
}

void loop() {
  int sensorValue = analogRead(pulsePin);

  // Update the moving average
  total -= readings[readIndex]; // Subtract the last reading
  readings[readIndex] = sensorValue; // Add the new reading
  total += readings[readIndex]; // Add the new reading
  readIndex = (readIndex + 1) % numReadings; // Move to the next index
  average = total / numReadings; // Calculate the average

  // Establish baseline if not already done
  if (!baselineEstablished) {
    baselineValue = average; // Set baseline value
    baselineEstablished = true; // Mark baseline as established
    delay(5000); // Wait for 5 seconds to establish baseline
  }

  // Simple thresholding to detect beats
  if (average > baselineValue + threshold) {
    if (millis() - lastBeatTime > 300) { // Debounce for 300ms
      lastBeatTime = millis();
      beatCount++;
    }
  }

  // Calculate heart rate every second
```

```
  if (millis() - lastTime >= 1000) {
    heartRate = beatCount * 60; // Convert beats to BPM
    beatCount = 0; // Reset beat count for the next second
    lastTime = millis();

    // Display heart rate on LCD
    lcd.setCursor(0, 0);
    lcd.print("Heart Rate: ");
    lcd.print(heartRate);
    lcd.print(" bpm");

    // Check if heart rate exceeds threshold
    if (heartRate > threshold) {
      digitalWrite(ledPin, HIGH); // Turn on LED
    } else {
      digitalWrite(ledPin, LOW); // Turn off LED
    }

    // Print to Serial Monitor for debugging
    Serial.print("Raw Value: ");
    Serial.print(sensorValue);
    Serial.print(" Average: ");
    Serial.println(average);
  }

  delay(10); // Short delay to avoid overwhelming the loop
}
```

Challenges Faced :

1. Component Placement and Wiring:

Arranging all sensors and components on the breadboard without creating a mess was challenging. Proper placement was essential to avoid loose connections.

2. Sensor Calibration:

The pulse sensor required calibration to set a proper threshold value.

3. Signal Interference:

The pulse sensor occasionally gave false triggers due to ambient light interference. Proper positioning and testing were necessary to minimize this issue.

4. Power Supply Management:

Ensuring the Arduino received sufficient power via USB while supporting all connected components was crucial. Any instability could lead to malfunctioning sensors.

5. Code Debugging:

Initial issues with incorrect pin assignments and logic errors in the code caused unexpected results. Systematic debugging and using the serial monitor helped fix them.

6. Circuit Diagram Understanding:

Designing a clear and neat circuit diagram that everyone could understand took effort. Ensuring all connections were accurate and easy to follow was essential.

7. Breadboard Limitations:

Limited space on the breadboard made it challenging to accommodate all connections neatly. Proper use of the positive and negative rails was crucial.

8. Component Sensitivity:

The pulse sensor's sensitivity needed adjustment to avoid false detections, especially in a well-lit environment.

Applications:

1. Healthcare & Medical Applications:-

Used in hospitals and home healthcare for continuous heart rate tracking.

2. Fitness & Sports Applications:-

Used in gyms and fitness centers to help individuals track their heart rate during exercise.

3.Stress and Mental Health Monitoring:-

Continuous heart rate monitoring can detect stress levels based on heart rate variability (HRV)**.**

4. IoT-Based Smart Healthcare Systems:

Heart rate data can be stored on **a** cloud database for long-term analysis..

5. Research and Biomedical Engineering:

Used in the development of low-cost heart rate monitoring devices.

6. Emergency and Safety Systems: Can be programmed to send emergency alerts (SMS or call) if heart rate drops too low or spikes too high**..**

Future scope:

1. Remote Health Monitoring:

Patients' heart rate data can be stored in cloud databases for doctors and caregivers to monitor remotely.

2. Smart Home & IoT Integration:

Heart rate data can be used to trigger smart home devices (e.g., adjust room temperature based on stress levels)..

3. Development of Low-Cost Wearable Heart Rate Monitors:

Future versions of this system can be miniaturized and embedded into wearable devices like smart watches or fitness bands.

4 .Cardiac Disease Prediction & Prevention:

Using continuous heart rate monitoring to detect and prevent cardiovascular diseases like arrhythmia or heart attacks.

5. Driver Fatigue and Drowsiness Detection:

Use Wi-Fi or Bluetooth modules for wireless communication, eliminating the need for wired connections.

Conclusion:

The Arduino-based heart rate detection system provides a cost-effective, real-time, and efficient solution for monitoring heart rate using a pulse sensor. This project demonstrates the practical application of embedded systems in healthcare and fitness monitoring, making heart rate detection accessible to individuals, athletes, and medical professionals.

By integrating Arduino with a pulse sensor, LCD display, and alert mechanisms (LED/Buzzer), the system successfully measures and displays heart rate (BPM) while providing alerts for abnormal values. This technology can be further enhanced with IoT, AI, and mobile app integration for remote monitoring and data analysis.

The system has a wide range of applications, including patient monitoring, fitness tracking, stress detection, and emergency alert systems. With future advancements, such as AI-based predictive health analysis, cloud connectivity, and wearable integration, this project has the potential to evolve into a fully automated and smart health monitoring system.

In conclusion, Arduino-based heart rate monitoring serves as an affordable, customizable, and scalable solution for both personal and professional healthcare applications, paving the way for more advanced biomedical engineering innovations.

Would you like me to add references, future enhancements, or project limitations.